
StudySync Resilience Report

Analysis & Redesign of a Distributed System

Parallel and Distributed Computing – Spring 2024

Date: May 10, 2026

PART 1: ANALYSIS OF THE CURRENT ISSUES

Problem 1 – Synchronization (Lost Update)

What happens? Two users load the same document at the same time, edit locally, and both send updates. The last write overwrites the first user's changes (lost update).

Root Cause: The API performs a read-modify-write with no concurrency control. The database update does not check a version (no optimistic locking) and does not lock rows, so it becomes last-write-wins.

Where it occurs: Between the API read and the subsequent write, and in the SQL update statement (e.g., `UPDATE docs SET content=? WHERE id=?`) that lacks a `WHERE version=?` guard.

Problem 2 – Coordination (Webhook Failure)

What happens? When a user cancels premium, Clerk sends a webhook. If the request is dropped during a network blip or a crash, the downgrade is never applied and the user stays premium.

Root Cause: The webhook receiver is not reliable: no persisted inbox, no idempotency key handling, no retries/queue, and no reconciliation to re-sync with Clerk.

Problem 3 – Fault Tolerance (Synchronous LLM Call)

What happens? The external LLM sometimes takes a long time to timeout. If the API waits on that call, FastAPI workers block. Under load, all workers get stuck and the app looks hung.

Root Cause: Tight coupling to an unreliable dependency without protective patterns (fast timeouts, circuit breaker, fallback). Slow dependency *cascades* to the whole service.

PART 2: PROPOSED ARCHITECTURAL IMPROVEMENTS

Solution 1 – Optimistic Locking for Synchronization

Add a **version** field to each document. Clients must send the version they read when attempting an update (e.g., **If-Match** or an explicit **version** field). The database performs a conditional update:

```
UPDATE docs SET content=?, version=version+1 WHERE id=? AND version=?;
```

If zero rows are updated, return **412 Precondition Failed** (or **409 Conflict**) with the latest content+version so the client can rebase and retry.

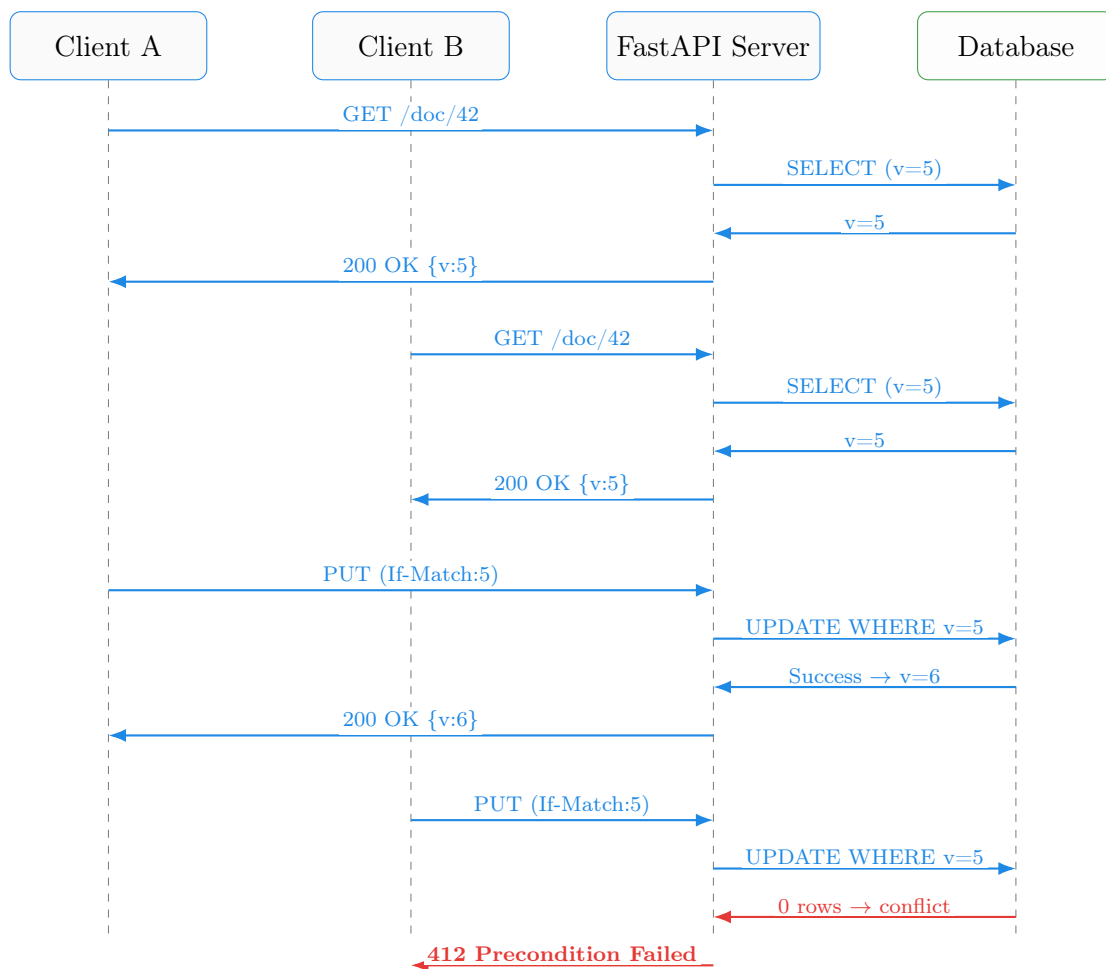


Figure 1: **UML Sequence Diagram: Optimistic Locking – Handling Concurrent Document Edits**

Solution 2 – Fault-Tolerant Webhook Handling

Use Clerk's `event_id` as an idempotency key. Store each event in an *inbox* table with a unique constraint and process it exactly-once *effectively* (idempotent handler). Run processing via a queue (e.g., Redis/RabbitMQ) with retries and a Dead-Letter Queue (DLQ). Add a periodic reconciliation job that compares local subscription state with Clerk and repairs missed changes.

Solution 3 – Circuit Breaker for LLM Calls

Implement a Circuit Breaker around the LLM call with short timeouts. After repeated failures, the circuit opens and the API fails fast, returning a fallback response (degraded but available). After a cooldown, allow limited probe requests (half-open) to decide whether to close the circuit.

CAP Theorem Trade-offs

- **Optimistic Locking:** prioritizes **Consistency** over write availability (conflicts return 412/409).
- **Webhooks:** prioritizes correctness via retries + reconciliation (eventual consistency).
- **Circuit Breaker:** prioritizes **Availability** and low latency with a fallback response.

We accept temporary conflicts or degraded responses to avoid global downtime.

Note: For Part 3, the Circuit Breaker pattern is implemented in the FastAPI code.